Introduction to Programming - Fall 2026

Working with text, then teaching your program to decide 🧠

Agenda

01	String operations, methods & f-strings
02	Exercise – Profile card
03	Comparison & logical operators
04	Exercise – Checks
–	Break (15 min)
05	If, elif, else, nesting & conditional expressions
06	Exercise – Weather advisor
07	Closing exercise – Weather quiz

Recap from last week

Last week we covered:

- 🐍 **Python basics** – `print()`, comments, running a script
- 📦 **Variables & data types** – `str`, `int`, `float`, `bool`
- ➕ **Arithmetic** – all the operators and how to use them
- ⌨️ **Input & output** – `input()`, `print()`, type conversion with `int()` and `float()`

Today we go deeper on **text**, then teach our programs to **make decisions**.

01 - String Operations

A **string** is text. In Python it lives between quotes: `"hello"` or `'hello'`.

Strings have their own operations – they are not the same as maths:

```
first = "Hello"
second = "World"

print(first + " " + second)    # Hello World   – concatenation
print("ha" * 3)                # hahaha         – repetition
print(len("Python"))           # 6              – length
```

`+` joins strings. `*` repeats a string. `len()` counts characters, including spaces.

String Operations in Practice

```
city = "Copenhagen"
country = "Denmark"

print(city + ", " + country)     # Copenhagen, Denmark
print(len(city))                 # 10
print("-" * 12)                  # ------------

# You cannot add a string and a number
age = 30
# print("Age: " + age)           # TypeError!
print("Age: " + str(age))        # Age: 30
```

`str()` turns a number into text, so it can be joined with `+`.

⚠️ `"10" + "3"` is `"103"`, not `13`. If you want maths, convert with `int()` or `float()` first.

String Methods

Strings come with built-in **methods** – actions you call with a dot `.`:

```
name = "  alice  "

print(name.upper())                 #   ALICE     (spaces kept)
print(name.lower())                 #   alice     (spaces kept)
print(name.strip())                 # alice
print(name.strip().capitalize())    # Alice

greeting = "Hello, World!"
print(greeting.replace("World", "Python"))  # Hello, Python!
```

💡 Methods do **not** change the original variable. They return a **new** string.

Methods Don’t Change the Original

```
name = "  alice  "
clean = name.strip().capitalize()

print(name)     #   alice     — still the same, spaces and all
print(clean)    # Alice       — the new string
```

If you want to keep the result, store it:

```
answer = input("yes or no? ")
answer = answer.strip().lower()
```

`.strip()` removes spaces the user typed by accident. `.lower()` makes `"Yes"`, `"YES"`, and `"yes"` all become `"yes"`.

Useful Methods for User Input

Method	What it does	Example
`.lower()`	all lowercase	`"Yes".lower()` → `"yes"`
`.upper()`	all uppercase	`"ok".upper()` → `"OK"`
`.strip()`	remove spaces at the ends	`" hi ".strip()` → `"hi"`
`.capitalize()`	first letter uppercase	`"alice".capitalize()` → `"Alice"`
`.replace(a, b)`	swap one piece of text	`"hi-there".replace("-", " ")` → `"hi there"`
`.startswith()`	does it begin with this?	`"Python".startswith("Py")` → `True`
`.endswith()`	does it end with this?	`"notes.txt".endswith(".txt")` → `True`

String Formatting

Building sentences with `+` gets messy fast. **f-strings** are the clean way.

Put an `f` before the quote, and drop expressions inside `{}`:

```
name = "Alice"
age = 30
city = "Copenhagen"

print(f"My name is {name}, I am {age} years old.")
print(f"I live in {city}.")
print(f"Next year I will be {age + 1}.")
```

Output:

```
My name is Alice, I am 30 years old.
I live in Copenhagen.
Next year I will be 31.
```

f-Strings in Practice

You can put **any expression** (code that produces a value) inside `{}`. For example, variables, maths, and method calls:

```
name = "maria"
city = "berlin"
age = 30

print(f"Hi, {name}!")                              # variable
print(f"Next year you will be {age + 1}.")         # maths
print(f"You live in {city.capitalize()}.")         # method call
print(f"{name.upper()} is {age * 12} months old.") # combined
```

```
Hi, maria!
Next year you will be 31.
You live in Berlin.
MARIA is 360 months old.
```

f-strings are the **modern, recommended** way to format text in Python.

Multiline f-Strings

Use three quotes `"""` to write one f-string over several lines. The line breaks show up in the output too:

```
name = "Maria"
city = "Berlin"
hobby = "hiking"

print(f"""👤 {name}
📍 {city}
❤️ {hobby}""")
```

```
👤 Maria
📍 Berlin
❤️ hiking
```

💡 Close the `"""` at the end of the last line, or you get an extra empty line.

Exercise 1 – Profile Card · 20 min

Ask for a name, a city, and a hobby. Print a short profile.

🟢 Basic

- Clean the name with `.strip().capitalize()`
- Print three lines with f-strings
- Print how many letters are in the name (`len`)

🔵 **Advanced** – a full profile card

Also ask for a last name, birth year, and bio. Print a card with:

- the full name in capitals, underlined with `=`
- a username: lowercase, `_` instead of spaces
- the age, worked out from the birth year
- the city, hobby, and bio, each with an emoji

```
MARIA JENSEN
============
👤 @maria_jensen
🎂 30 years old
📍 Berlin
❤️ Hiking
📝 I love maps and coffee.
```

Copy emojis from getemoji.com

Exercise 1 – Hints

🟢 Basic

1. One `input()` per question. Store each answer.
2. `name = name.strip().capitalize()`
3. `print(f"Hi, my name is {name}.")`
4. `print(f"My name has {len(name)} letters.")`

🔵 Advanced

1. `"ha" * 3` gives `hahaha`. The `=` line has to fit **any** name, not just `MARIA JENSEN`.
2. `"2026-10-03".replace("-", "/")` gives `2026/10/03`, which is handy for the username.
3. Build the username and the line first, then print everything with one `"""` f-string.
4. Test with messy input like `" anna maria "`. Does everything still line up?

From Text to Decisions

So far every line runs, top to bottom, every time.

Next we teach the program to **choose**:

- Is this password correct?
- Is the user old enough?
- Did they type `"yes"`, or something else?

To choose, Python needs to **compare** values – numbers and strings – and get back `True` or `False`.

03 - Comparison Operators

Comparison operators **compare two values** and always return `True` or `False`:

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 3`	`True`
`>`	Greater than	`10 > 3`	`True`
`<`	Less than	`2 < 1`	`False`
`>=`	Greater than or equal	`5 >= 5`	`True`

Comparing Numbers and Text

```
age = 20
print(age >= 18)              # True

password = "abc123"
print(password == "abc123")   # True
print(password == "ABC123")   # False — case sensitive!

city = "Copenhagen"
print(city.lower() == "copenhagen")   # True
print(len(city) > 5)                  # True
```

⚠️ Don't confuse `=` (store a value) with `==` (compare two values).

Strings compare **character by character**. `"Yes"` and `"yes"` are not equal.

Is It Inside? The `in` Operator

`in` checks whether a piece of text appears **anywhere** inside a string:

```
email = "maria.jensen@example.com"

print("@" in email)                 # True
print("jensen" in email)            # True
print("gmail" in email)             # False
print("JENSEN" in email)            # False — case sensitive!
print("JENSEN".lower() in email)    # True
```

`not in` checks the opposite:

```
password = "sunny2026"
print("123" not in password)        # True
```

Logical Operators

Logical operators let you **combine multiple conditions**:

Operator	Meaning	Result is `True` when…
`and`	Both must be true	**Both** conditions are `True`
`or`	At least one must be true	**Either** condition is `True`
`not`	Reverses the result	The condition is `False`

Logical Operators in Practice

```
age = 25
has_ticket = True
city = "aarhus"

print(age >= 18 and has_ticket)          # True
print(age < 18 or has_ticket)            # True
print(not has_ticket)                    # False

print(city == "copenhagen" or city == "aarhus")   # True
```

Clean user input first, then combine the checks:

```
answer = input("Do you have a ticket? ").strip().lower()
print(answer == "yes" or answer == "y")
```

Exercise 2 – Checks · 20 min

🟢 **Basic** – player sign-up

Store a player's `age`, `password`, `city`, and `points` in variables. Print `True` or `False` for each:

- `age` is at least 18
- `password` is `"secret"`
- `city` is `"Copenhagen"`
- `points` is not 0

Example: `age = 20` → `True`

🔵 **Advanced** – password checker

Players must have strong passwords for their account. Write a program that reads a username and a password with `input()` and prints `True` if the password is strong, or `False` otherwise. A password is strong if it:

- has at least 8 characters
- does not start with `123`
- is not `password`, in any case
- does not contain the username, in any case

Example: `Sunny2026` → `True` · `Maria2026` for user `maria` → `False`

Exercise 2 – Hints

🟢 Basic

```
age = 20
password = "secret"
city = "Copenhagen"
points = 10
```

Then `print(age >= 18)`, `print(password == "secret")`, and so on.

🔵 Advanced

1. `"Python".startswith("Py")` gives `True`. Which operator flips it?
2. Lowercase both before comparing, so `Password` and `MARIA2026` are caught too.
3. Store each rule in a variable, like `long_enough = len(password) >= 8`, then combine them with `and`.

QUESTIONS BEFORE THE BREAK?

Break (15 min)

05 - If Statements

An `if` runs code **only when a condition is** `True`:

```
temperature = 30

if temperature >= 25:
    print("T-shirt weather ☀️")

print("Have a nice day!")
```

- The condition ends with a colon `:`
- The indented line runs only if the condition is `True`
- `"Have a nice day!"` is not indented, so it **always** runs

Indentation Decides What's Inside

The lines indented under the `if` are its **block**. Moving one line changes the program:

```
temperature = 10

if temperature >= 25:
    print("T-shirt weather ☀️")
    print("Bring sunscreen 🧴")
print("Have a nice day!")
```

```
Have a nice day!
```

```
temperature = 10

if temperature >= 25:
    print("T-shirt weather ☀️")
print("Bring sunscreen 🧴")
print("Have a nice day!")
```

```
Bring sunscreen 🧴
Have a nice day!
```

- **Tab** indents, **Shift + Tab** removes the indent. VS Code uses **4 spaces**
- VS Code also indents for you when you press Enter after a `:`
- No indentation after `:` gives `IndentationError: expected an indented block`

Adding Else

`else` runs when the condition is `False`:

```
weather = input("What's the weather like? ").strip().lower()

if weather == "rain":
    print("🌂 Take an umbrella!")
else:
    print("😎 Leave the umbrella at home.")
```

Only **one** of these two lines will ever print – never both.

Adding Elif

`elif` ("else if") adds more choices in between:

```
weather = input("What's the weather like? ").strip().lower()

if weather == "rain":
    print("🌂 Take an umbrella!")
elif weather == "snow":
    print("🧤 Wear gloves and boots!")
elif weather == "sun":
    print("🧴 Don't forget sunscreen!")
else:
    print("🤔 Hmm, I don't know that weather.")
```

Python checks the conditions **from top to bottom** and runs only the first one that is `True`. If none of them match, the `else` block runs.

You can add as many `elif` as you need, and `else` is optional.

Combining Conditions

Logical operators and string methods belong **inside** `if` statements:

```
weather = input("What's the weather like? ").strip().lower()
wind = input("Is it windy? (yes/no) ").strip().lower()

if weather == "rain" and wind == "yes":
    print("🧥 Wear a raincoat, an umbrella will flip!")
elif weather == "rain" or weather == "snow":
    print("🌂 Take an umbrella!")
else:
    print("😎 No umbrella needed.")
```

Order matters: windy rain is checked **before** plain rain, otherwise the umbrella would always win.

Nested Conditionals

You can put an `if` **inside** another `if` – this is called nesting:

```
weather = input("What's the weather like? ").strip().lower()

if weather == "snow":
    driving = input("Do you have to drive? (yes/no) ").strip().lower()
    if driving == "yes":
        print("🚗 Leave early, the roads are slippery!")
    else:
        print("🛷 Go sledding!")
else:
    print("🙂 No snow plans needed today.")
```

The inner `if` only runs when the outer one is `True`, so we only ask about driving when it snows.

Truthy & Falsy Values

In Python, **every value has a boolean meaning** – not just `True` and `False`.

These values are treated as `False`:

```
False    # the boolean False
0        # zero
0.0      # zero as a float
""       # empty string
```

Everything else is treated as `True` – any non-zero number, any non-empty string, etc.

Truthy & Falsy in Practice

An empty string is falsy, so this checks "did they type anything?":

```
city = input("Which city are you in? ").strip()

if city:
    print(f"Checking the weather in {city.capitalize()}...")
else:
    print("You didn't enter a city.")
```

```
rain_mm = int(input("How many mm of rain today? "))

if rain_mm:
    print(f"{rain_mm} mm of rain today. 💧")
else:
    print("No rain today! ☀️")
```

💡 This is a very **Pythonic** pattern – you'll see it everywhere in real code.

Conditional Expressions

An `if` statement chooses which **code** runs. A conditional expression chooses which **value** to use:

```
temperature = 30
clothes = "a T-shirt" if temperature >= 25 else "a jacket"
print(f"Wear {clothes}.")    # Wear a T-shirt.
```

The pattern is: `value_if_true if condition else value_if_false`

It gives the same result as this longer `if` / `else`:

```
if temperature >= 25:
    clothes = "a T-shirt"
else:
    clothes = "a jacket"
```

Conditional Expressions in Practice

```
temperature = int(input("How many degrees is it? "))
feel = "warm" if temperature >= 20 else "cold"
print(f"It's {feel} outside.")

city = input("Your city: ").strip()
place = city.capitalize() if city else "your area"
print(f"Here's the forecast for {place}.")
```

⚠️ Use it to **pick between two values**. To run different code, use a normal `if` / `else`.

Predict the Output 🤔

What does this program print? Discuss with your neighbour for a minute.

```
weather = "snow"
temperature = 2
wind = "yes"

if weather == "rain" or weather == "snow":
    if temperature < 0 and wind == "yes":
        print("🥶 Stay inside!")
    elif wind == "yes":
        print("🧥 Wear a hood, it's windy!")
    else:
        print("🌂 Take an umbrella!")
elif temperature < 0:
    print("🧤 Wear gloves!")
else:
    print("😎 Enjoy your day!")
```

And with `weather = "sun"` and `temperature = -5`?

Exercise 3 – Weather Advisor · 20 min

🟢 **Basic** – what to wear

Ask for the temperature and whether it's raining (`yes` / `no`). Print what to wear:

- 20° or more → `Wear a T-shirt.`
- 10° to 19° → `Wear a jacket.`
- below 10° → `Wear a warm coat.`
- raining → also `Take an umbrella.`

Example: `14` , `yes`

```
Wear a jacket.
Take an umbrella.
```

🔵 **Advanced** – daily report

Read the weather, temperature, and wind (km/h). Print a report like the example:

- wind 75+: a storm warning instead of advice
- wind 30+: feels 5° colder, dress for that
- rain or snow: umbrella, or raincoat if windy
- rain or snow below 0°: `Icy roads!`

Example: `" RAIN "` , `-1` , `40`

```
Rain, -1°C, wind 40 km/h
Feels like -6°C
Wear a warm coat.
Wear a raincoat.
Icy roads!
```

Exercise 3 – Hints

🟢 Basic

1. The temperature is a number, so convert it with `int()`.
2. Check from the warmest down: `>= 20` first, then `>= 10`, then `else`.
3. The umbrella is a separate `if` **after** the outfit, so both lines can print.

🔵 Advanced

1. Which rule stops all the others? Make it the outer `if`, and put the rest in its `else`.
2. Work out the feels-like temperature **once**, before you choose clothes.
3. Rain gear and icy roads can both print. Do they belong in the same `if` / `elif` chain?

Closing Exercise – Weather Quiz · 20 min

🟢 **Basic** – three questions

Build a weather quiz. Start `score` at `0`, add `1` for each correct answer, and print the total at the end.

- At what °C does water freeze? → `0`
- What falls from clouds as frozen flakes? → `snow`
- What appears when the sun shines during rain? → `rainbow`

Example: `You scored 2 out of 3.`

🔵 **Advanced** – a fairer quiz

Players complain the quiz marks right answers as wrong. Make it fairer:

- `" Snow "` and `"SNOW"` count as correct
- answers like `"a rainbow"` or `"snowflakes"` count too
- add a fourth weather question of your own
- after the score, print a different message for 4/4, 3/4, and below

Example: `You scored 4 out of 4.` then `Perfect forecast! ☀️`

Closing Exercise – Hints

🟢 Basic

1. `score = 0` before the questions.
2. After each answer, an `if` checks it. Inside, indented: `score = score + 1`
3. The freezing question needs `int(input(...))`.
4. One f-string at the end for the score.

🔵 Advanced

1. Clean every text answer before you check it.
2. Look back at the `in` operator slide.
3. Use `if` / `elif` / `else` on the score for the final message.

WEEK 4 – STRINGS & CONDITIONALS

What We Learned Today

✅ **String operations** – concatenation with `+`, repetition with `*`, `len()`

✅ **String methods** – `.lower()`, `.strip()`, `.capitalize()`, `.replace()`

✅ **f-strings** – `f"Hello, {name}!"` to build clean output

✅ **Comparison operators** – `==`, `!=`, `<`, `>`, `<=`, `>=`, and `in` to look inside text

✅ **Logical operators** – `and`, `or`, `not`

✅ **if / elif / else** – and nested conditionals

✅ **Truthy & falsy** – an empty string is `False`

✅ **Conditional expressions** – `value if condition else other` to pick a value

Homework

🏠 Finish the weather quiz if you didn't, then extend it:

🟢 **Basic** – add 2 more questions and keep the score correct (`out of 5`).

🔵 **Advanced** – turn it into a game show:

- Wrong answer? Print a hint and give **one** more try.
- 2 points on the first try, 1 on the second.
- All answers right on the first try? Unlock a **bonus question**.

Example:

```
Frozen flakes? rain
Hint: it's white and cold.
Try again: snow
+1 point
```

📌 **Remember:** Commit your work to GitHub when you're done!



See you next week!